CS528 Homework 4

HTTP file service and Pub/Sub reporting on VMs

Prem Rajendran | Boston University ID: U99248729

October 13, 2026

Course repository: `https://github.com/Pr3mR4jendran/BU-CS528-Cloud-Computing`

Submission status. Implementation, functional evidence, the full stress experiment, and automatic cleanup are verified. The HW-4 folder is prepared for the course repository; publishing the local commits and recording the actual billed cost remain before submission. Section 14 records the cost evidence still needed.

1 Architecture and configuration

The HW3 file service is recreated as a threaded Python HTTP server on a small Compute Engine VM. GET uses the URL path and POST uses the request payload to select an existing HTML object. A separate VM consumes forbidden-country events from Pub/Sub and prints them to stdout. The instructor's Linux client runs on a third VM.

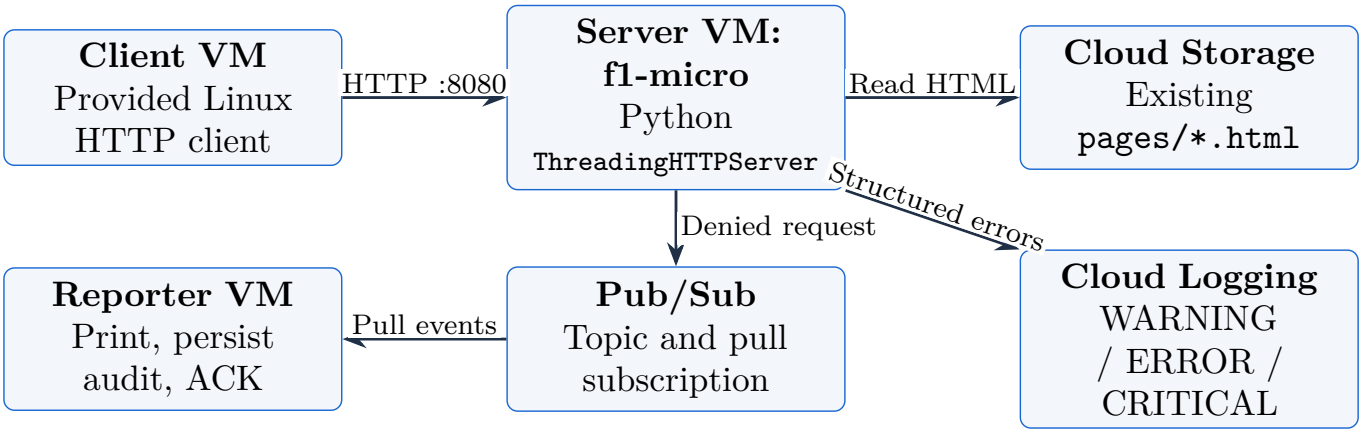


Figure 1: Three separate VMs. The reporter also writes a deduplicated audit object in the existing bucket under the run's private audit prefix.

Setting	Measured deployment
Project / location	`bu-cs528-prem-project`; `us-central1-a`, `us-central1`
Dataset	`cs528-hw3-bu-cs528-prem-project-1791250771`; `pages/0.html` through `pages/11999.html`
Server	`cs528-hw4-server`: f1-micro, Debian 12, Python 3.11
Reporter / client	`cs528-hw4-reporter`: e2-micro, Debian 12; `cs528-hw4-client`: e2-micro, Ubuntu 24.04
Historical endpoint	`http://104.197.255.22:8080/pages/0.html`
Network	Dedicated VPC/subnet; HTTP to server on TCP 8080; IAP SSH on TCP 22

The server address was reserved/static during the experiment. All HW4 infrastructure has since been deleted; this historical endpoint is no longer a live demonstration. A new deployment obtains its own address. Ubuntu 24.04 is used for the client because the supplied binary requires `GLIBC_2.38`; Debian 12 is retained for both services.

2 Reproducing the deployment

Clone the course repository, enter `HW-4`, and place the instructor-provided x86-64 Linux `http-client` binary beside `deploy.py`. It is excluded from Git. Install Python and Google Cloud CLI locally. The dataset bucket must already contain the homework HTML files.

```
git clone https://github.com/Pr3mR4jendran/BU-CS528-Cloud-Computing.git
cd BU-CS528-Cloud-Computing/HW-4
gcloud auth login
Copy-Item .\config.example.json .\config.json
python -m venv .venv
& .\.venv\Scripts\python.exe -m pip install -r .\requirements.txt
& .\.venv\Scripts\python.exe -m unittest discover -s tests -v
```

Edit `config.json` to set the signed-in provisioning account, project, bucket and dataset prefix. The measured deployment used:

```
{
  "projectId": "bu-cs528-prem-project",
  "account": "premraj@bu.edu",
  "bucketName": "cs528-hw3-bu-cs528-prem-project-1791250771",
  "objectPrefix": "pages/", "maxFileIndex": 11999,
  "region": "us-central1", "zone": "us-central1-a",
  "prefix": "cs528-hw4", "port": 8080,
  "serverMachine": "f1-micro", "reporterMachine": "e2-micro",
  "clientMachine": "e2-micro",
  "clientImageFamily": "ubuntu-2404-lts-amd64",
  "clientImageProject": "ubuntu-os-cloud",
  "httpSourceRanges": ["0.0.0.0/0"]
}
```

The provisioning user must be able to enable project APIs, administer Compute resources, create service accounts, attach them to VMs, change the relevant IAM policies, administer Pub/Sub, and read the existing dataset. The measured provisioning account had project Owner. A grader with narrower access also needs the corresponding Compute and service account permissions and IAP-secured Tunnel User for SSH. The script uses instance metadata SSH access with OS Login disabled; an organization requiring OS Login needs a deployment adjustment. These are provisioning privileges, separate from the limited VM runtime roles.

```
& .\.venv\Scripts\python.exe .\deploy.py plan
& .\.venv\Scripts\python.exe .\deploy.py up
if ($LASTEXITCODE -ne 0) { throw 'Deployment incomplete' }
$hw4 = Get-Content .\state.json -Raw | ConvertFrom-Json
$hw4.serverUrl
& .\.venv\Scripts\python.exe .\deploy.py status
```

The single `up` command creates all three VMs and starts both application services. It enables the required Compute, IAM, Storage, Pub/Sub, Logging, IAP and Resource Manager APIs, reserves the server IP, applies scoped IAM, and waits for server/reporter readiness. Generated startup scripts embed the application source and configuration in VM metadata, install dependencies, and enable systemd units. First boot needs no GitHub credentials. An incomplete deployment can be resumed with the same code/config and state file.

3 Runtime identities and least privilege

The server and reporter use different dedicated service accounts. Their credentials come from the VM metadata service; application key files are unnecessary. Both applications run as the unprivileged Linux user `hw4app`. The client VM has no attached service account.

Identity	Granted role	Resource / restriction
Server	Storage Object Viewer	Existing bucket, conditional read of `pages/` objects only
Server	Pub/Sub Publisher	`cs528-hw4-forbidden` topic only
Server	Logs Writer	Project, for the application's structured error logs
Reporter	Pub/Sub Subscriber	`cs528-hw4-reporter` subscription only
Reporter	Storage Object User	Existing bucket, conditional access to `hw4-logs/<owner>/` only
Client	None	No attached account or application IAM grant

The account names are `cs528-hw4-server@bu-cs528-prem-project.iam.gserviceaccount.com` and `cs528-hw4-reporter@bu-cs528-prem-project.iam.gserviceaccount.com`. The cloud-platform OAuth scope permits API calls; the IAM bindings above limit authorization. The exact bucket IAM conditions are:

```
// Server: roles/storage.objectViewer
resource.name.startsWith('projects/_/buckets/cs528-hw3-bu-cs528-prem-project-1791250771/objects/pages/')

// Reporter: roles/storage.objectUser
resource.name.startsWith('projects/_/buckets/cs528-hw3-bu-cs528-prem-project-1791250771/objects/hw4-logs/
    rf6136407ec914afdbeb03a8733430b0b/')
```

The server cannot write dataset objects. The reporter's audit access does not extend to the dataset prefix. Storage Object User is scoped to the audit prefix to support a generation-checked read/modify/write and retry on concurrent updates.

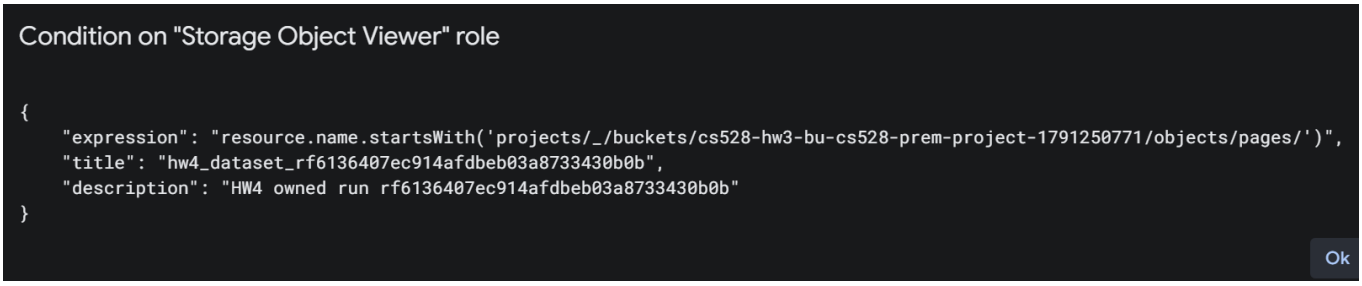


Figure 2: Server bucket role condition restricting object reads to the dataset prefix.



Figure 3: Reporter bucket role condition restricting audit access to this run's owned prefix. The conditions are also transcribed above for legibility.

Exported policies and VM identities/scopes were verified before cleanup. The appendix shows project Logs Writer and resource-level Pub/Sub roles. Cleanup removed the runtime grants.

4 VM placement, static IP and automatic startup

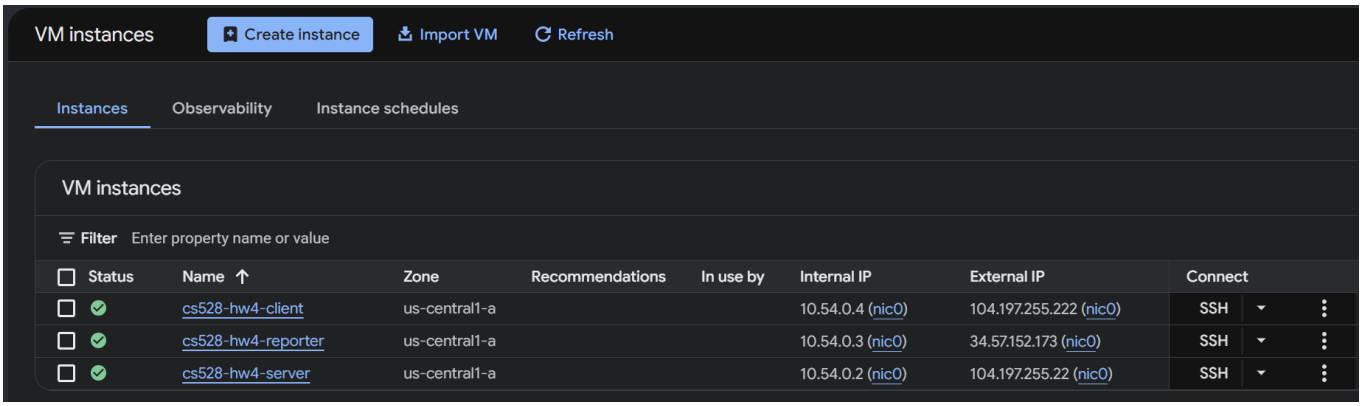


Figure 4: All three VMs running in us-central1-a. The client and reporter are distinct from the HTTP server.

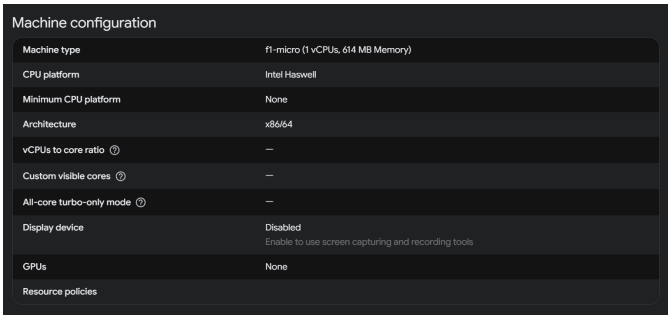


Figure 5: Server machine configuration: f1-micro.

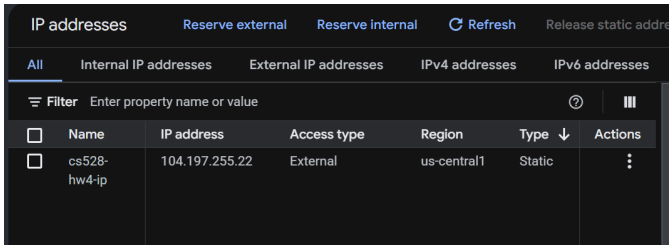


Figure 6: Reserved static address attached to the server.

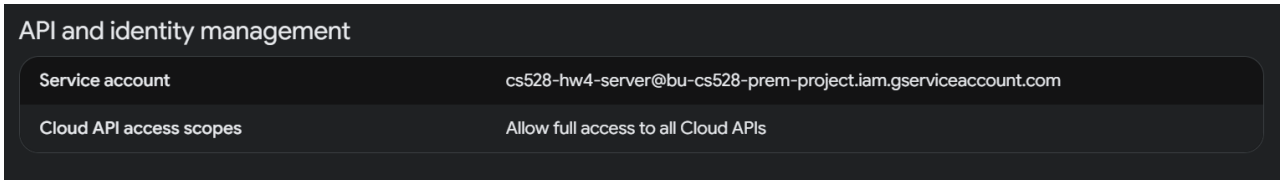


Figure 7: Dedicated server service account attached to the server VM.

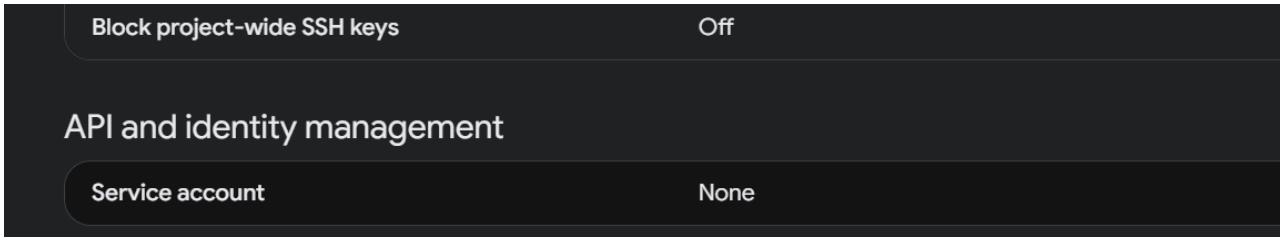


Figure 8: Client VM identity panel: no attached service account. The complete screenshot is retained in the source package.

Startup creates `cs528-hw4-server.service` and `cs528-hw4-reporter.service`, both enabled and active. Units depend on network readiness, restart on failure after five seconds, and use `NoNewPrivileges`, `PrivateTmp`, `ProtectSystem=strict` and `ProtectHome`. Source and runtime configuration live in `/opt/cs528-hw4`.

5 HTTP behavior and curl verification

Request	Status	Severity	Observed behavior
GET existing file	200	–	Original HTML bytes from Cloud Storage
POST filename payload	200	–	Same original HTML bytes
GET missing object	404	WARNING	Structured `file_not_found` record
PUT, DELETE, HEAD, CONNECT, OPTIONS, TRACE, PATCH	501	ERROR	Structured `unsupported_method` record
GET with banned X-country	400	CRITICAL	Denial log plus Pub/Sub event; no object download

Country denial follows the assignment's simulated list: North Korea, Iran, Cuba, Myanmar, Iraq, Libya, Sudan, Zimbabwe and Syria. It uses the supplied `X-country` request header. The list is an assignment policy, not a claim about current export regulations.

The `demos.py` harness ran actual curl requests and saved response headers, bodies, exit codes and request event IDs. All 19 cases passed: GET, POST, one missing file, all seven unsupported methods and all nine forbidden countries. All 17 expected error records were matched to their Cloud Logging severities. GET and POST body hashes matched the bucket object's SHA-256:

```
c3be7cacaad7e54bac9e5050419002b0545418e31094933b4239813e2a6f2b88
```

Run the full demo after deployment; use the new URL from state rather than the historical IP:

```
& .\.venv\Scripts\python.exe .\demos.py
$hw4 = Get-Content .\state.json -Raw | ConvertFrom-Json
$base = $hw4.serverUrl -replace '/pages/0.html$',''
curl.exe -i "$base/pages/12000.html"
curl.exe -i -X PATCH "$base/pages/0.html"
curl.exe -i -H 'X-country: Iran' "$base/pages/0.html"
```

POST uses a JSON body such as `{"filename":"pages/0.html"}` with `Content-Type: application/json`; form and plain-text payloads are also supported.

```
PS C:\Users\premr\Desktop\BU Coursework\Fall 2026\CS 528 - Cloud Computing\Assignments\HW-4> curl.exe -i "$($hw4.serverUrl)/pages/12000.html"
HTTP/1.1 404 Not Found
Server: BaseHTTP/0.6 Python/3.11.2
Date: Tue, 13 Oct 2026 07:23:41 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 16
X-Request-Id: 036bf548-3ddc-4fe7-b9a0-17e845a192eb
Connection: close

File not found.
PS C:\Users\premr\Desktop\BU Coursework\Fall 2026\CS 528 - Cloud Computing\Assignments\HW-4> curl.exe -i -X PATCH "$($hw4.serverUrl)/pages/0.html"
HTTP/1.1 501 Not Implemented
Server: BaseHTTP/0.6 Python/3.11.2
Date: Tue, 13 Oct 2026 07:23:45 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 32
X-Request-Id: 63513a57-abb6-497d-9db7-dad9916b3e19
Connection: close

HTTP method is not implemented.
PS C:\Users\premr\Desktop\BU Coursework\Fall 2026\CS 528 - Cloud Computing\Assignments\HW-4> curl.exe -i -H 'X-country: Iran' "$($hw4.serverUrl)/pages/0.html"
HTTP/1.1 400 Bad Request
Server: BaseHTTP/0.6 Python/3.11.2
Date: Tue, 13 Oct 2026 07:23:50 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 19
X-Request-Id: f8f6844d-1977-4c42-b01a-f8c129e3dec3
Connection: close

Permission denied.
```

Figure 9: Saved curl responses: missing file 404, unsupported PATCH 501 and Iran country denial 400.

6 Browser demonstrations

The browser first loaded the original HTML page from the server. From that same origin, DevTools `fetch` requests exercised GET/POST 200, missing-file 404, PATCH 501 and Iran 400. The console shows the actual returned statuses; its red 404/501/400 messages are the expected error cases, rather than a failed test harness.

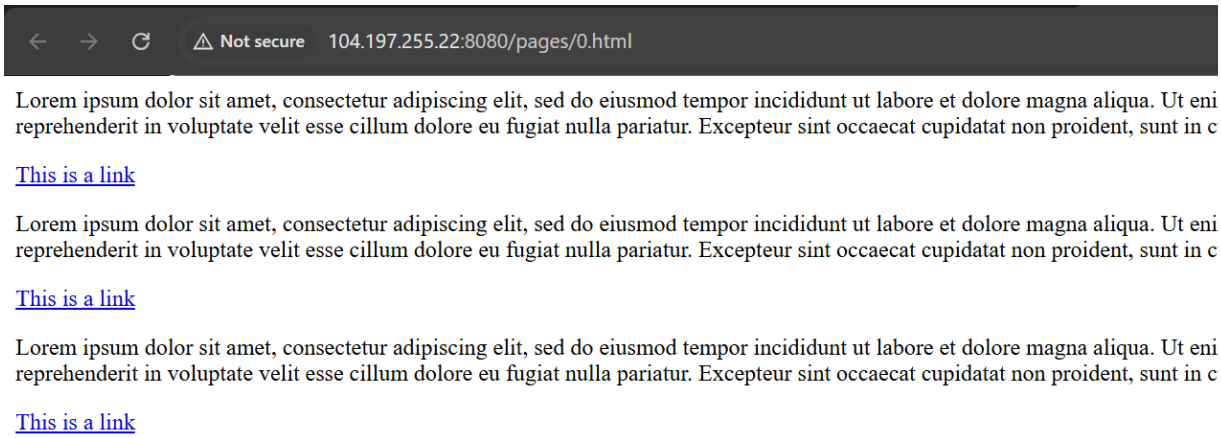


Figure 10: Upper-left browser excerpt: HTML rendered at the server's historical static-IP endpoint. The complete screenshot is retained with the source.

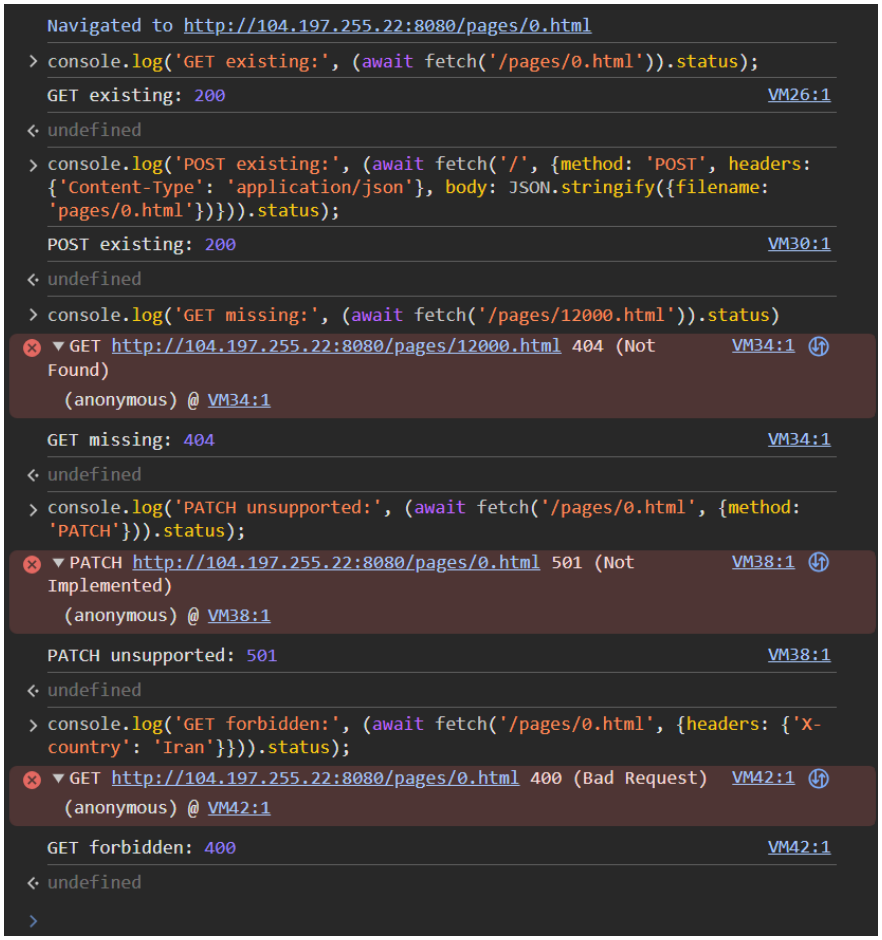


Figure 11: One browser request for each status case, including both supported methods. GET and POST return 200; the remaining requests return 404, 501 and 400.

7 Structured Cloud Logging: missing and unsupported requests

The application calls Cloud Logging with a structured JSON payload and an explicit severity. Records carry `event_id`, method, path, filename, status, country and timestamp. The two screenshots below show the complete error payload and severity in Logs Explorer.

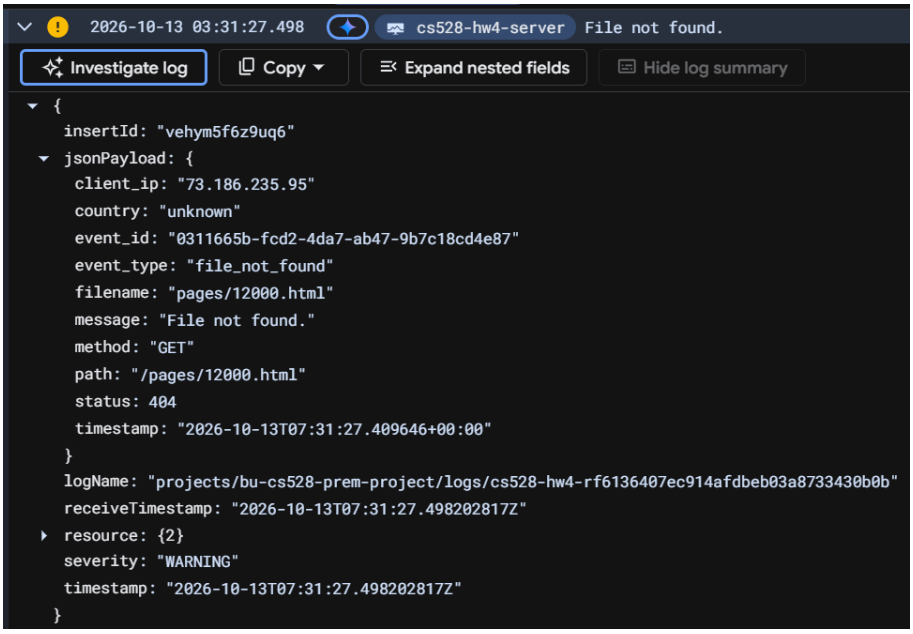


Figure 12: Missing `pages/12000.html`: status 404, event type `file_not_found`, severity WARNING.

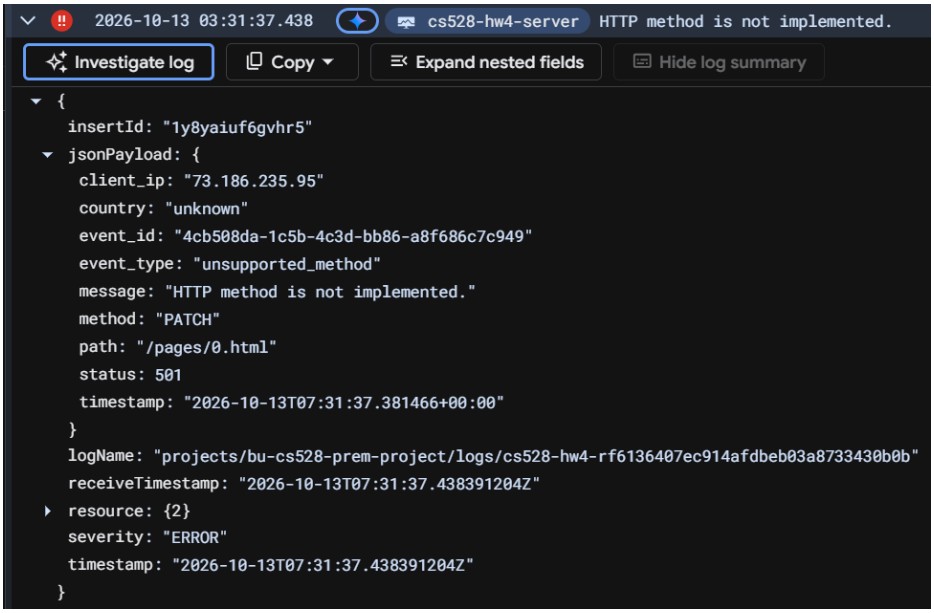


Figure 13: Unsupported PATCH: status 501, event type `unsupported_method`, severity ERROR.

The full pre-cleanup export contains 1,833 custom application log entries. The final collection was not restricted to the first 1,000 entries. These exported records survive locally even though cleanup deletes this run's custom cloud log.

8 Forbidden-country events and reporter delivery

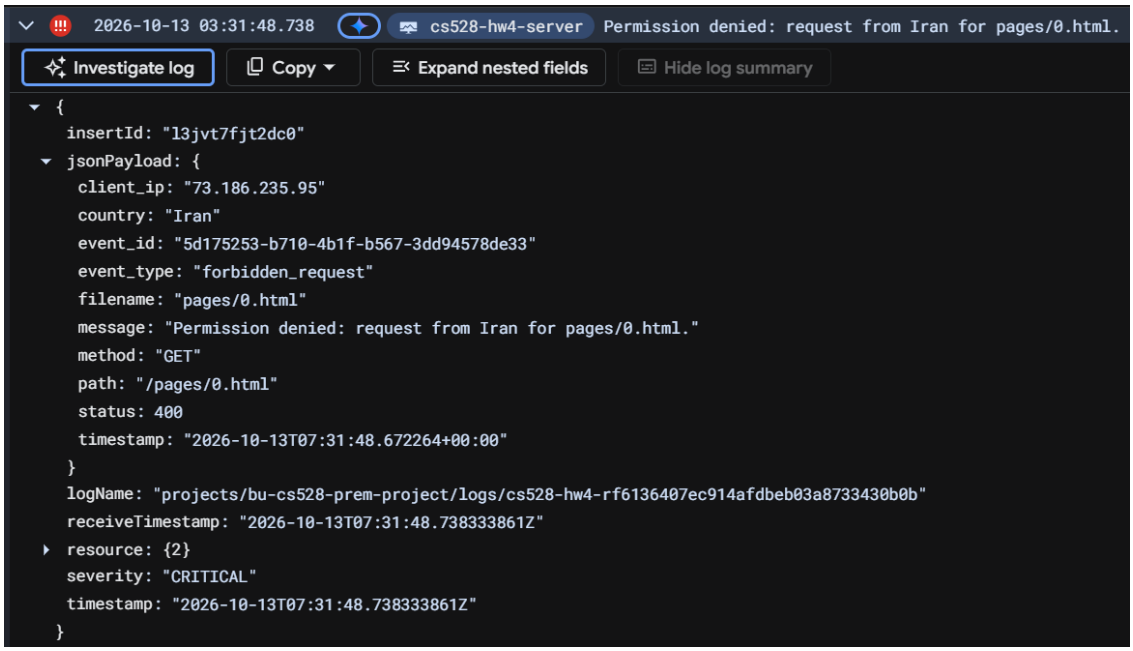


Figure 14: An Iran request is denied with HTTP 400 and a structured CRITICAL Cloud Logging entry.

For this example, the same event ID appears in the CRITICAL log, the reporter's stdout and the persisted audit object. The reporter prints an appropriate denial message with the country and filename. Its stdout record uses WARNING; the server's Cloud Logging record uses the assignment-required CRITICAL severity.

```
event_id: 5d175253-b710-4b1f-b567-3dd94578de33
country: Iran                    filename: pages/0.html
message: Permission denied: request from Iran for pages/0.html.
pubsub_message_id: 22010629669199521
```

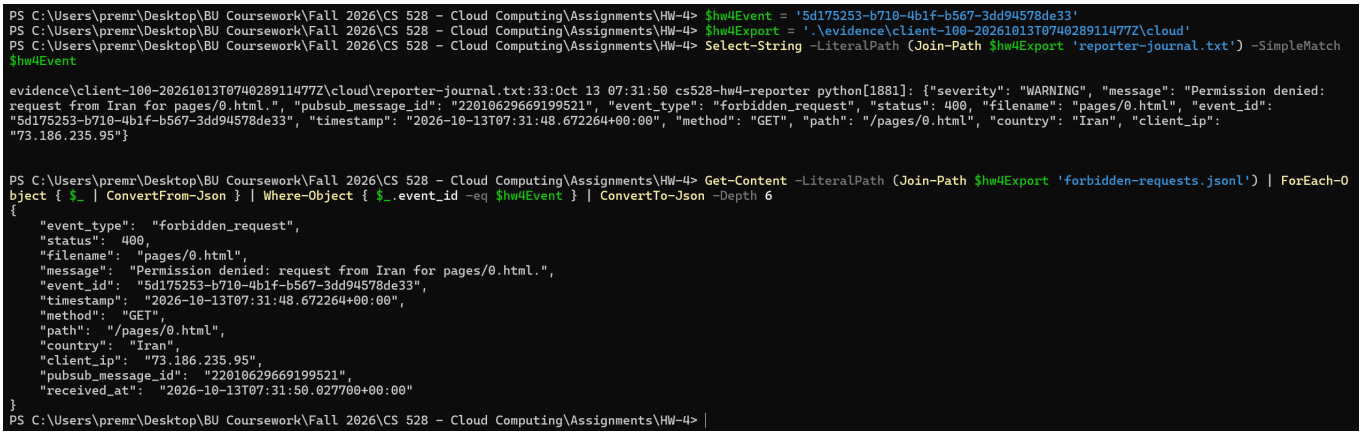


Figure 15: Saved reporter journal and audit-object lookup for the same event ID. The details are transcribed above for readability.

The subscriber acknowledges a message only after the audit write and stdout record. Audit updates use Cloud Storage generation preconditions and deduplicate by event ID, so a redelivery after an interrupted acknowledgement does not append a second entry. Before cleanup, all **1,119** forbidden events from the final stress suite matched CRITICAL server logs, reporter stdout and exactly one audit entry. The full retained audit has 1,783 unique events, including earlier demonstrations and trial runs.

9 Provided client and service readiness

`run_client.py` copies/runs the instructor's binary on `cs528-hw4-client`, independent of both service VMs. The verified standalone run requested 100 files with the following remote command:

```
/opt/cs528-hw4/http-client -d 104.197.255.22 -b none -w pages \
  -n 100 -i 11999 -p 8080 -r 0 -t 60 -v

# Local orchestration after up:
& .\.venv\Scripts\python.exe .\run_client.py
```

Check	Latest standalone 100-request result
Expected / observed responses	100 / 100
Successful downloads	94 HTTP 200 responses
Expected country denials	6 HTTP 400 responses; all 6 persisted denials verified
Client exit / verification	Exit code 0; `passed=true`

The 100-request evidence includes compact client output, request IDs, body/type checks, the command, and a cloud export. Expected country denials are not server-capacity errors. The result is saved under `client-100-20261013T074028911477Z`.

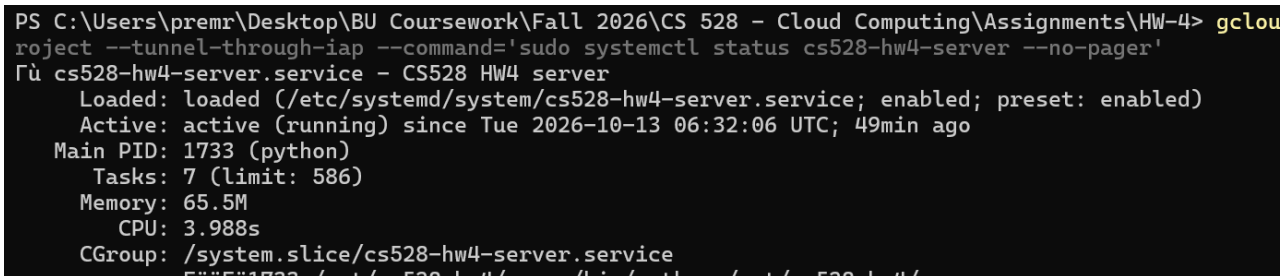


Figure 16: Server startup/status excerpt: the systemd unit is enabled and active. The complete original screenshot is retained in the source package.

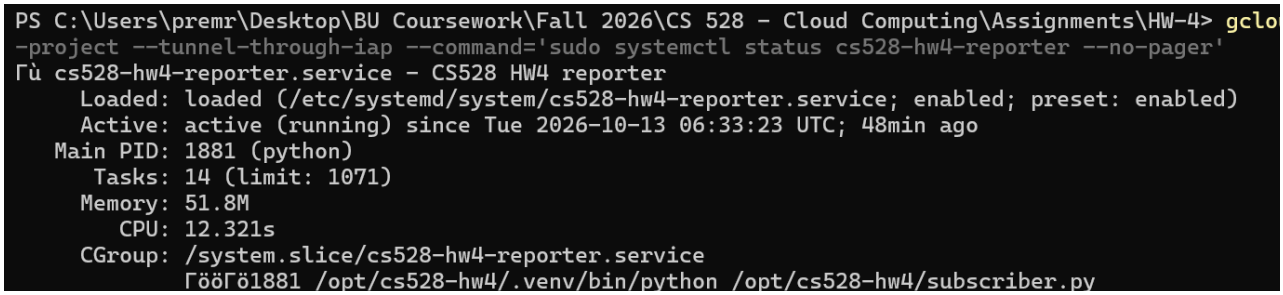


Figure 17: Reporter startup/status excerpt: its own systemd unit is enabled and active. Figure 15 shows received forbidden requests on stdout.

The deployment command's readiness checks include a real bucket-backed HTTP request, the reporter's ready message, client bootstrap completion and the supplied binary's help output. The local test suite also passes all 33 tests, covering real HTTP sockets, audit behavior, cleanup ownership, sampling and failure accounting. Cloud IAM and deployment success are supported by live exports, not inferred from mocked local tests.

10 Stress-test method and measured results

The report uses only the completed `stress-20261013T155600814186Z` experiment. One, two, three and four concurrent provided-client processes each ran for **180 seconds** on the client VM, separated by 15-second cooldowns. Each process makes sequential requests; "clients" describes processes rather than a guaranteed number of in-flight requests. Each slot has the same seed across load levels. Owned process groups stop at the deadline; the script rejects more than four clients before contacting the cloud.

```
& .\.venv\Scripts\python.exe -m pip install -r .\requirements-performance.txt
& .\.venv\Scripts\python.exe .\run_stress.py --seconds 180 --counts 1 2 3 4
& .\.venv\Scripts\python.exe .\plot_stress.py --latest
```

Server and client Linux counters were sampled at approximately one-second intervals. Every retained client sample shows the requested active process count. Per stage, 179 server intervals cover about 179.14 seconds wholly within the 180-second workload; boundary-straddling intervals are excluded. CPU excludes idle, iowait and steal, and host memory is `MemTotal-MemAvailable`. Python RSS is reported separately. NIC RX/TX use the default interface and include storage/API traffic as well as HTTP traffic. Means are weighted by interval duration. Peaks are sampled interval peaks, not instantaneous hardware maxima. Mbit/s uses decimal megabits; MiB uses binary mebibytes.

Clients	Seconds	Responses	HTTP 200	Expected 400	Responses/s
1	180	2,643	2,521	122	14.68
2	180	4,841	4,620	221	26.89
3	180	7,257	6,936	321	40.32
4	180	10,633	10,178	455	59.07

Table 1: All four stages completed. Unexpected HTTP errors, invalid responses, transport failures, request timeouts and early client exits were zero at every level.

Clients	CPU (%) mean / peak	Host memory mean / peak MiB	Python RSS peak MiB	RX mean / peak Mbit/s	TX mean / peak Mbit/s
1	7.59 / 19.39	372.79 / 378.59	91.32	9.27 / 17.28	9.27 / 17.24
2	13.73 / 25.53	368.32 / 374.91	92.75	17.13 / 30.11	17.13 / 30.14
3	19.50 / 28.42	369.28 / 371.48	94.11	25.67 / 39.26	25.66 / 38.14
4	26.99 / 35.05	374.78 / 377.80	93.05	37.54 / 51.72	37.59 / 51.76

Table 2: Measured server resource usage. The four-client row supplies the required three-minute peak resource measurements.

Across the suite, 25,374 responses comprise 24,255 successful downloads and 1,119 expected denials. No capacity-related failure was observed up to the assignment's four-client limit. This establishes successful service at the tested offered load, rather than a maximum server capacity. Full verbose HTML bodies were drained/discarded on the client VM; stdout timestamps are observations, so no per-request latency claim is made.

11 Performance graphs and scaling

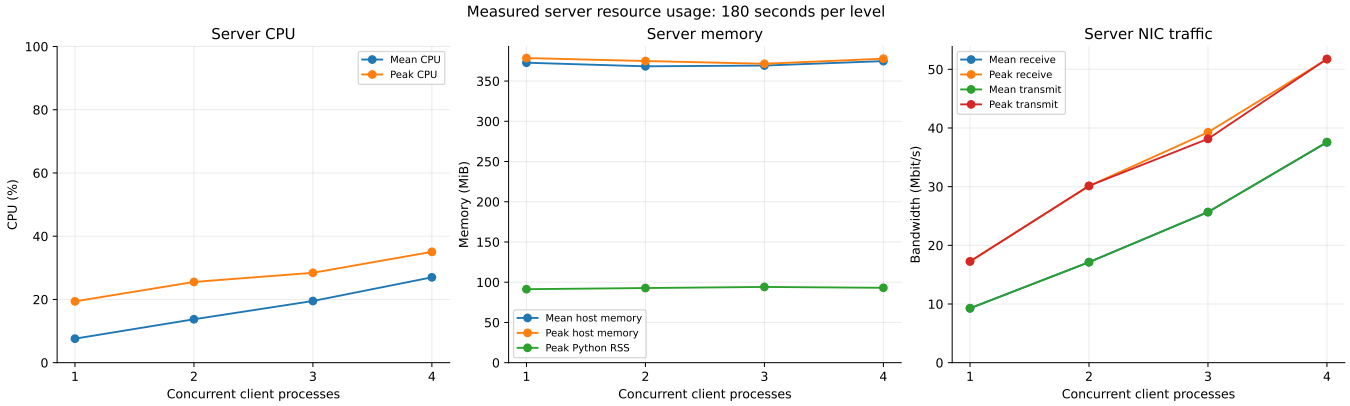


Figure 18: Server CPU, host memory and NIC bandwidth by concurrent client count, using the four completed 180-second workloads. Mean values are interval-weighted; peaks are sampled interval maxima.

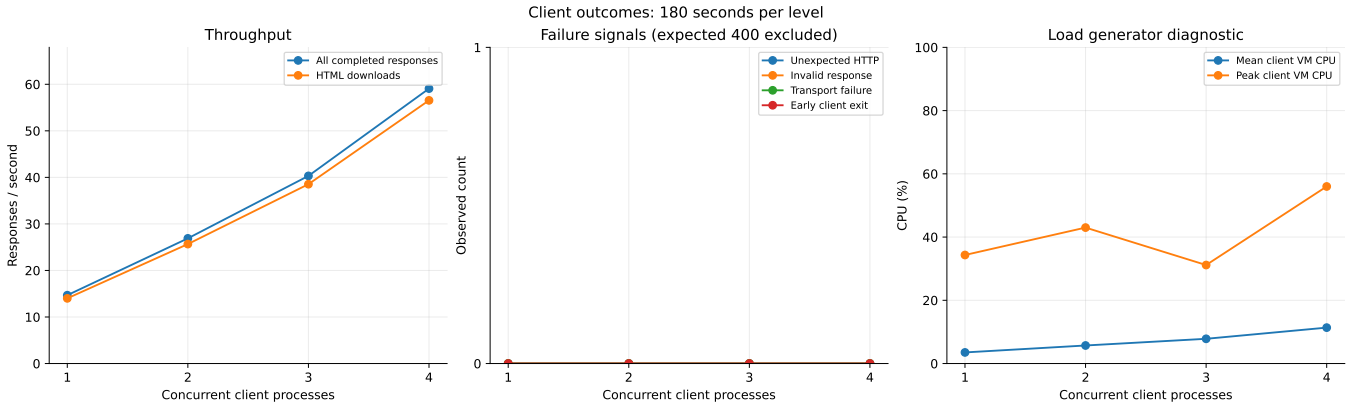


Figure 19: Throughput grows with client count. The separate failure panel excludes expected HTTP 400 denials and shows zero unexpected failures at every tested level.

CPU. Mean server CPU grows from 7.59% to 26.99%, about 3.56 times, while response throughput grows about 4.02 times. The threaded server can overlap network waits for Cloud Storage downloads. These measurements are consistent with substantial I/O waiting and spare CPU at the tested load; they do not independently identify a limiting resource.

Memory. Mean host usage remains roughly 368–375 MiB, and Python peak RSS remains about 91–94 MiB. Additional request threads and temporary response data do not create a sustained rise in the sampled working set for one through four clients.

Bandwidth. Mean NIC transmit rate rises from 9.27 to 37.59 Mbit/s, about 4.05 times; receive rate behaves similarly. Successful requests fetch HTML from Cloud Storage and send it to the client, explaining the similar RX/TX rates and near-throughput scaling. Random file sizes, request mix and storage/API overhead explain small deviations.

12 Resource time series and experiment limits



Figure 20: One-second sampled server CPU, host memory and transmit bandwidth during each three-minute workload. Each trace is aligned to its own stage start.

At four clients, sampled server peaks are **35.05% CPU**, **377.80 MiB host memory**, **93.05 MiB Python RSS**, **51.72 Mbit/s RX** and **51.76 Mbit/s TX**. Client mean CPU reaches 11.36% at four clients, with a 56% sampled peak.

The results concern this client binary, object set, region and short experiment. All clients share an e2-micro VM, and workload instrumentation, network/API latency, varying object sizes and burstable VM behavior can influence the offered load. A lack of errors at four processes does not prove that more clients would also succeed. The assignment prohibits that extension. Earlier short smoke trials and an interrupted run are excluded from the graphs and capacity conclusion. The interrupted run revealed client-timeout parsing behavior; the final parser records timeouts explicitly and resynchronizes without suppressing a failure signal.

13 Automatic deletion and final verification

The instructor clarified that deletion must be available automatically. The same deployment driver supplies that option:

```
& .\.venv\Scripts\python.exe .\deploy.py collect
& .\.venv\Scripts\python.exe .\deploy.py down
if ($LASTEXITCODE -ne 0) { throw 'Cleanup incomplete; keep state.json' }
& .\.venv\Scripts\python.exe .\deploy.py status
```

`down` exports evidence first, checks ownership, removes tracked VMs and their owned boot disks, removes IAM grants, deletes Pub/Sub resources, releases the reserved IP, deletes firewall rules/subnet/VPC/service accounts, and removes this run's audit object and custom application log. It verifies resource absence and reports failure if anything remains. A foreign ownership marker prevents deletion. An audit export failure does not leave chargeable infrastructure running; it retains the audit and reports incomplete cleanup.

The first cleanup deleted infrastructure but its final check encountered inconsistent IAM list/describe views after a successful service-account deletion. The corrected verifier makes up to four checks, with 1/2/4-second delays, after a confirmed deletion. It still requires authorized proof of absence and does not hide persistent permission errors. The recovery run demonstrates safe repeated cleanup of already-absent resources.

```
// down-20261013T163845931615Z/cleanup.json
{
  "status": "deleted",
  "remaining": [], "failures": [],
  "verifiedAt": "2026-10-13T16:40:19.533943+00:00"
}
// Additional read-only check, 2026-10-13T16:41:52.344298Z
tracked_resource_count: 15
runtime_bucket_and_project_grants_removed: true
audit_object_absent: true
custom_cloud_log_entries_absent: true
original_dataset_object_present: true
local_evidence_preserved: true
```

The 15 tracked resources are three VMs, three boot disks, two service accounts, one VPC, one subnet, two firewall rules, one reserved address, one topic and one subscription. The original bucket/dataset, earlier homework resources, enabled project APIs, provider audit logs, SSH credentials and local evidence are shared or retained prerequisites. They are intentionally preserved. `state.json` records `status=deleted`.

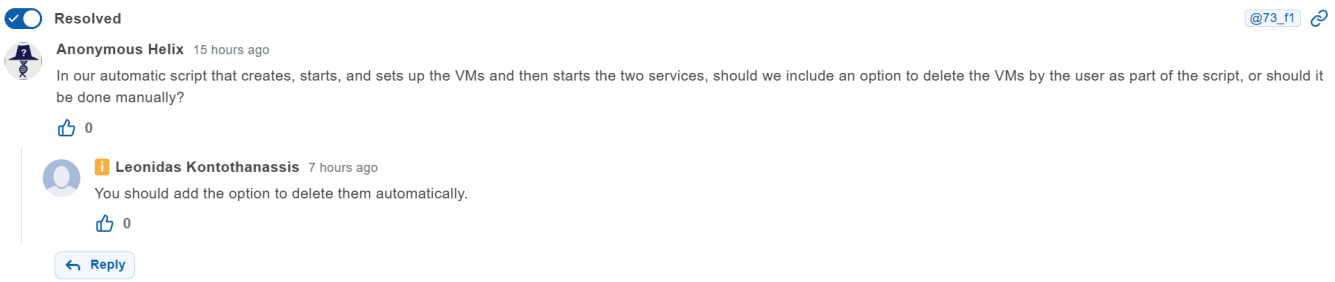


Figure 21: Instructor clarification requiring an automatic VM deletion option. The implemented `down` action provides that option and verifies its result.

A fresh `up` can provision from code, but a post-cleanup delete/recreate cloud experiment was not performed and is not claimed here. Read the new state URL on any subsequent run.

14 Cost accounting and submission artifacts

Actual billed cost: awaiting Cloud Billing evidence. The retained records contain deployment and cleanup timestamps but no Billing export. An actual money amount cannot be derived reliably from resource metrics or a zero currently displayed before billing settles. The cost requirement remains incomplete until the filtered Billing report is recorded.

Event (October 13, 2026)	UTC	America/New_York
Deployment state created	06:28:26	02:28:26 EDT
Services / client ready	06:49:12	02:49:12 EDT
Final stress window	15:56:36–16:09:22	11:56:36–12:09:22 EDT
First cleanup began	16:29:22	12:29:22 EDT
Cleanup absence verified	16:40:19	12:40:19 EDT
Additional absence checks	16:41:52	12:41:52 EDT

State timestamps bound the workflow; VM creation/deletion records and Billing determine actual billable time. The later verification retry did not recreate or extend the life of resources already removed in the first cleanup.

To finish cost reporting, select the project's Billing Reports view for October 13, group by Service and then SKU, and save the chart/table, currency, usage charges, credits and net subtotal. Include Compute VM usage, disks, external IPv4, network transfer and the API/storage services represented in the report. Download the corresponding CSVs. The experiment shares a project and dataset with earlier homework: label-filtered `assignment=cs528-hw4` usage can help attribute labeled resources, but may omit unlabeled/shared charges. Report project-day totals and clearly explain any HW4 allocation instead of silently equating the whole project bill with the stress test alone.

Google states that costs typically appear within a day but sometimes take more than 24 hours. Record the capture time and distinguish usage charges before credits from the net amount paid after credits. Documentation: `https://docs.cloud.google.com/billing/docs/how-to/reports`. The local `BILLING-EVIDENCE.md` provides the capture checklist.

Code and report package

The local course-repository `HW-4` folder contains both services, the deployment/cleanup driver, demo/client/stress tools, pinned requirements, portable example configuration, tests, run guides and this report's source/assets/PDF. The intended publication location is: `https://github.com/Pr3mR4jendran/BU-CS528-Cloud-Computing/tree/main/HW-4`. Local commits still need to be pushed before that location can be used for grading. The instructor binary, actual config/state, virtual environment and full raw evidence are retained locally and excluded from the public source folder. Report evidence summaries and the selected screenshots/plots are included with the report.

The full stress record contains command logs, raw server/client metrics, compact client output, completion status, summary JSON/CSV and graph files. `MANIFEST.sha256` verifies the archived experiment files. Pre-cleanup verification ties the stress denials to cloud logs/stdout/audit, and post-cleanup verification records resource absence.

A Pub/Sub and project IAM evidence

The following portions of the saved console screenshots show the actual resource-level role assignments. The complete screenshots are included in the report source package. Exported policies were also checked before cleanup; these screenshots describe the historical running deployment.

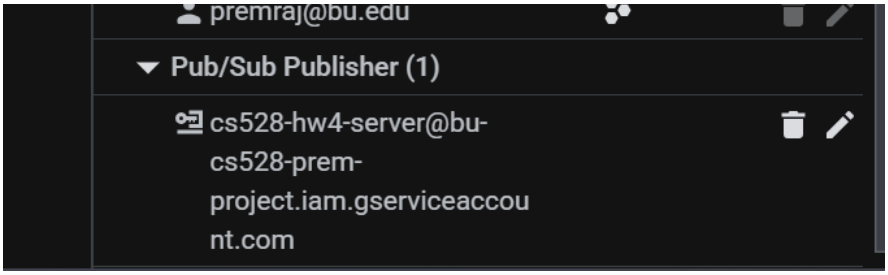


Figure 22: Topic IAM panel: the server account has Pub/Sub Publisher.

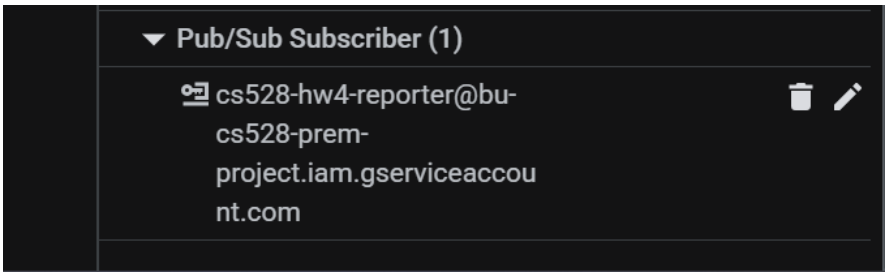


Figure 23: Subscription IAM panel: the reporter account has Pub/Sub Subscriber.

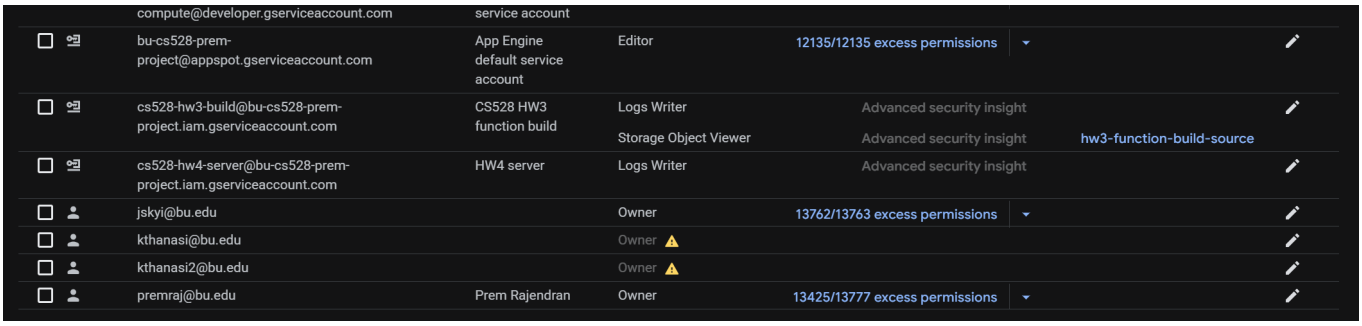


Figure 24: Project IAM rows for the dedicated service accounts, including the server's Logs Writer role. Runtime roles are summarized in Section 3.

B Evidence inventory and validation

Evidence set	Retained artifact / interpretation
HTTP demonstrations	`demos-20261013T072213446007Z`: 19 request cases, body hashes, 17 error-severity matches
Standalone client	`client-100-20261013T074028911477Z`: 100 responses, 94 downloads, 6 verified denials
Final stress suite	`stress-20261013T155600814186Z`: 180 seconds each at 1–4 clients; report-ready, measurement-valid, complete
Pre-cleanup collection	`collect-20261013T161443335686Z`: full logs/journals/audit and event correlation checks
Cleanup recovery	`down-20261013T163845931615Z`: no remaining resources/failures; additional absence/grant checks
Local code checks	All 33 HTTP, audit, deployment and stress-accounting tests passed
Screenshots	VM/static-IP/identities; conditional bucket IAM; Pub/Sub/project roles; service startup; curl/browser/logs; reporter correlation
Billing	No actual billed-cost export available when this report was generated

The code package includes compact verification summaries under `report/evidence`. The full raw exports remain in the working assignment directory. Report figures use the completed full experiment; no interrupted or smoke-run data are merged into those plots. All charts are included as vector PDFs, and the original screenshots are retained beside the TeX source. Upload the complete source ZIP when compiling on Overleaf so those assets are available.

Useful commands after cloning

The application libraries are installed automatically by startup scripts on the service VMs. Installing the local requirements supports tests and demo validation. Graphing additionally uses `requirements-performance.txt`. To rebuild the report, use:

```
# Run inside report/ with the assets directory present:
tectonic CS528-HW4-Report.tex
# Or compile twice with pdfLaTeX on Overleaf/a TeX installation.

# Application lifecycle from HW-4/ (after editing config.json):
python deploy.py up
python demos.py
python run_client.py
python run_stress.py --seconds 180 --counts 1 2 3 4
python plot_stress.py --latest
python deploy.py collect
python deploy.py down
```

The `up`/`down` actions remain the reproducible entry points for grading. They derive account/resource names from the chosen prefix and project, embed source at boot, preserve shared dataset objects and save per-run evidence before deletion.